# CineMatch: Movie Recommendation System — Case Study

## Executive summary

CineMatch is a Streamlit web application that recommends movies with three different techniques and measures how well each one works. It was built as a Data Warehousing and Mining (DWM) mini-project on the public MovieLens dataset.

The project takes raw rating files through a data-warehousing pipeline (extract, clean, load, mine, evaluate, visualize) and lets a user pick a model, choose a user, and see personalised recommendations. A model-comparison dashboard scores all approaches on the same held-out ratings using 28 metrics and 16 graphs, so the choice of model rests on evidence rather than on intuition.

**Headline result:** *[fill in after running the app: which model had the best average rank, and by how much it beat the random baseline]*.

## Background and problem statement

Streaming catalogues hold thousands of titles, and users give up when they cannot find something worth watching. Recommender systems solve this by predicting which unseen titles a person is most likely to enjoy.

The problem for this project: **help users discover movies using ratings and genre metadata, and show objectively which recommendation approach works best.** Two questions drive the work:

1. How can ratings and genres be turned into useful personalised suggestions?
2. How do we compare very different methods fairly, on accuracy, ranking quality, variety and speed?

For the DWM course, the project also demonstrates the full data-mining workflow: collecting data, cleaning it, building analytical tables, applying mining algorithms, and presenting evaluated results.

## Objectives and scope

The project aims to deliver a working, explainable recommender plus a rigorous model comparison.

- Load and clean the MovieLens ratings and movies tables.
- Implement three recommenders: popularity-based, content-based (genres) and collaborative filtering (user similarity).
- Offer an interactive dashboard: pick a model and a user, get top-N picks, or find movies similar to a chosen title.
- Explore the dataset: genre mix, rating distribution, highest-rated films.
- Evaluate every model on the same held-out data with error, classification, ranking, diversity and speed metrics, and present them as tables and graphs.

**Out of scope:** real-time serving, user accounts, deep-learning models, and Netflix data (see the dataset section).

## Dataset

The app uses **MovieLens Latest Small** from GroupLens (about 100,000 ratings from roughly 600 users on roughly 9,700 movies; the dashboard shows the exact counts on startup). It is a public research dataset, not Netflix viewing data. The app downloads it automatically on first run, or reads your own `data/movies.csv` and `data/ratings.csv`.

| Table | Columns | Role |
|---|---|---|
| movies.csv | movieId, title, genres | Movie metadata; genres are pipe-separated (e.g. Action\|Sci-Fi) |
| ratings.csv | userId, movieId, rating, timestamp | Explicit ratings from 0.5 to 5 in half-star steps |

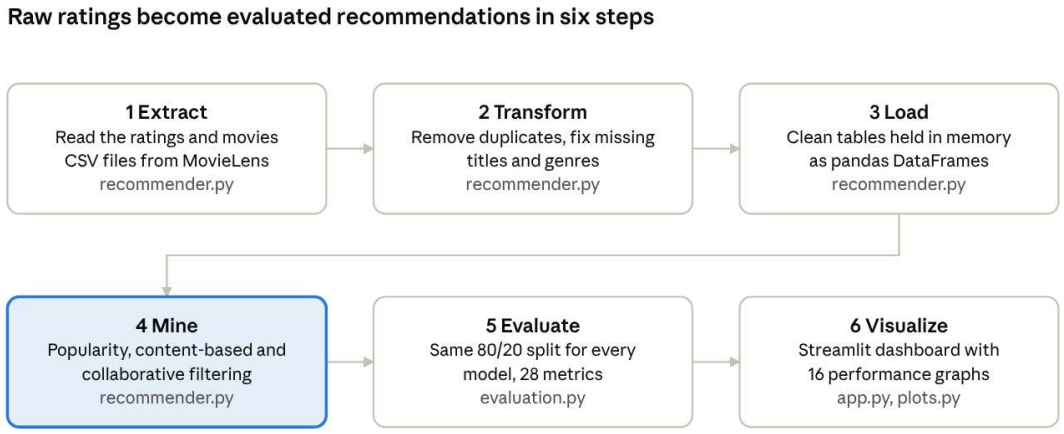
**Cleaning and transformation** (`clean_data` in `recommender.py`):

- Remove duplicate movies by movieId and duplicate user-movie ratings.
- Drop ratings with missing userId, movieId or rating.
- Fill missing titles with "Unknown title" and missing genres with an empty string.
- Aggregate ratings per movie (mean, count) and expand genres into one-hot columns for the models.

One limitation to state plainly: ratings are explicit scores given on the MovieLens site, so the data reflects what users chose to rate, not everything they watched.

## Solution architecture

CineMatch follows the classic data-warehousing and mining pipeline, with the mining step (highlighted) at its centre.




DWM pipeline · 6 steps, two rows

Data flows through steps 1 to 3 in the top row, then the mining, evaluation and visualization steps in the bottom row.

**Technology:** Python, pandas, NumPy, scikit-learn, Streamlit, Plotly (dashboard charts) and Matplotlib (performance graphs).

| File | Responsibility |
|---|---|
| app.py | Streamlit dashboard with four tabs: Recommendations, Model comparison, Dataset explorer, Project guide |
| recommender.py | Data loading and cleaning; popularity, content-based and collaborative recommenders; similar-movies search |
| evaluation.py | Per-model evaluation: predictions, top-K lists and all metrics |
| plots.py | The 16 performance graphs |
| run_evaluation.py | Command-line run that saves metric tables and graphs to the outputs folder |

## The recommendation models

Three techniques are implemented, from simplest to most personalised, plus a random baseline used only as a reference line.

| Model | Idea | How it scores a movie | Strength | Weakness |
|---|---|---|---|---|
| Popularity based | Recommend what most people like | Bayesian-smoothed average rating: (n / (n + m)) x movie mean + (m / (n + m)) x global mean, so films with few ratings are pulled toward the average | Fast, works for brand-new users | Same list for everyone; favours blockbusters |
| Content based (genres) | Recommend films like the ones this user liked | Builds a genre profile from the user's ratings; scores each unseen film by | Needs no other users; explainable | Only sees genres, so many films tie; narrow suggestions |

| Model | Idea | How it scores a movie | Strength | Weakness |
|---|---|---|---|---|
| | | how well its genres match | | |
| Collaborative filtering (user similarity) | People who rate like you will like what they liked | Cosine similarity between users on their rating vectors; weighted average of the 40 most similar users' ratings | Finds tastes genres cannot describe | Needs enough overlapping ratings; slower |
| Random baseline (reference only) | No intelligence | Global-average rating, random ranking | Shows what "no skill" looks like | Not a real recommender |

A fourth feature, **similar movies**, ranks other films by genre overlap with a chosen title, with average rating as a tie-breaker.

If a user has no liked films (content based) or is not in the matrix (collaborative), the app falls back to the popularity model so it always returns something.

## Evaluation methodology

Every model is tested on the **same random 80/20 split of the ratings** (fixed seed 42), and each model produces its own predictions and its own top-10 lists. A held-out rating of 4 or higher counts as "the user liked it".

Ranking lists exclude films the user rated in the training data, and ranking metrics are averaged over users who liked at least one held-out film. Metrics fall into six families:

| Family | Metrics | Question answered | Better |
|---|---|---|---|
| Rating prediction | RMSE, MAE, median error, R squared, bias, share within 0.5 and 1.0 stars | How close is the predicted rating to the real one? | Lower error, higher R squared |
| Like / dislike | Accuracy, precision, recall, F1, specificity, ROC-AUC, PR-AUC, confusion matrix (predicted rating of 3.5 or more = "like") | Can the model separate liked from not-liked films? | Higher |
| Top-K ranking | Precision@K, Recall@K, F1@K, NDCG@K, | Are the liked films near | Higher |

| Family | Metrics | Question answered | Better |
|---|---|---|---|
| | MAP@K, HitRate@K, MRR@K, curves for K = 1 to 20 | the top of the list? | |
| Beyond accuracy | Catalogue coverage, novelty, list diversity, long-tail share | Is the system varied, or does it repeat the same hits? | Higher means broader |
| Efficiency | Fit time, scoring time | How fast is it? | Lower |
| Segments | RMSE by user activity and by movie popularity | How does it behave for new users and rare films (cold start)? | Lower |

To summarise, the dashboard ranks the three models on ten core metrics (RMSE, MAE, R squared, F1, ROC-AUC, Precision@K, Recall@K, NDCG@K, MAP@K, HitRate@K) and reports the average rank and number of metrics won.

## Results

*These cells are left blank on purpose: the numbers must come from your own run on MovieLens. Click Run full model evaluation in the app, then copy values from the Tables tab or from* `outputs/model_metrics_full.csv`.

| Model | RMSE | MAE | F1 (like) | ROC-AUC | Precision@10 | Recall@10 | NDCG@10 | Coverage | Fit time (s) |
|---|---|---|---|---|---|---|---|---|---|
| Popularity based | | | | | | | | | |
| Content based (genres) | | | | | | | | | |
| Collaborative filtering | | | | | | | | | |
| Random baseline | | | | | | | | | |

**The 16 performance graphs** (saved to `outputs/graphs/` and shown in the dashboard). Insert each one you use, with a caption stating what it shows:

| Graph | What it shows |
|---|---|
| 01 RMSE and MAE | Typical size of the rating error for each model |
| 02 R squared and accuracy bands | How much rating variation is explained; share of guesses within half a star and one star |
| 03 Error distribution | Whether errors are tightly centred on zero or spread out |
| 04 Calibration and bias | Whether the model over- or under-predicts at each true rating |
| 05 Confusion matrices | Liked and disliked films classified correctly or not |
| 06 Classification metrics | Accuracy, precision, recall, F1, specificity, AUC side by side |
| 07 ROC and precision-recall curves | Trade-offs across every decision threshold |
| 08 Ranking metrics at K | Precision, recall, F1, NDCG, MAP, hit rate and MRR for the top-10 list |
| 09 Ranking curves vs K | How each ranking metric changes as the list grows from 1 to 20 |
| 10 Beyond accuracy | Coverage, novelty, diversity and long-tail share |
| 11 Fit and scoring time | Speed of each model |
| 12 RMSE by segment | Behaviour for light vs heavy users and rare vs popular films |
| 13 Radar chart | Relative strengths of the three models on one picture |
| 14 Heatmap | Every metric for every model, shaded by who is best |
| 15 Overall ranking | Average rank and metrics won |
| 16 Error vs ranking quality | Whether low rating error goes with good top-10 lists |

## Application screenshots

Capture these from the running app (`streamlit run app.py`) and paste them under the matching row. Each screenshot should include the full browser window so the tab names are visible.

| Figure | Where in the app | What to capture | Caption to write |
|---|---|---|---|

| Figure | Where in the app | What to capture | Caption to write |
|---|---|---|---|
| 1 | Top of the page | The four counter cards (movies, ratings, users, average rating) and the green "Dataset ready" banner | Dataset loaded and summarised |
| 2 | Recommendations tab, Popularity model | Picked user, the top-10 table and the score bar chart | Popularity-based recommendations |
| 3 | Recommendations tab, Content model | Same user with Content Based selected | Content-based recommendations |
| 4 | Recommendations tab, Collaborative model | Same user with Collaborative selected | Collaborative recommendations (compare with Figures 2 and 3) |
| 5 | Recommendations tab, similar movies | A chosen title with its similar-movies list | Find-similar feature |
| 6 | Model comparison tab, Tables | The all-metrics table and best-per-metric table | Full metric comparison |
| 7 | Model comparison tab, Rating error | RMSE/MAE and error-distribution graphs | Rating-prediction accuracy |
| 8 | Model comparison tab, Like / dislike | Confusion matrices, ROC and PR curves | Classification quality |
| 9 | Model comparison tab, Top-K ranking | Metrics at K and curves vs K | Ranking quality |
| 10 | Model comparison tab, Overall | Radar chart, heatmap and overall ranking | Overall comparison |
| 11 | Dataset explorer tab | Genre bar chart and rating histogram | Dataset exploration |

*[Insert screenshots here, one per figure.]*

## Analysis and insights

Use your own numbers to answer these questions; each is a known pattern in recommender systems, so check whether your results agree and say so either way.

- **Accuracy is not one number.** A model can have a good RMSE yet produce weak top-10 lists, because the two metrics test different skills. Graph 16 shows whether the two agree for your models.
- **Popularity is a strong baseline.** Because a few films receive most ratings, a plain popular-films list is often hard to beat on ranking metrics. Compare every model with the popularity bar and the random bar, not only with each other.
- **Personalisation should help where there is data.** Collaborative filtering has the most to gain for users and films with many ratings; graph 12 shows where it does and where it falls back.
- **Cold start.** Rare films (few training ratings) and light users give every model trouble. Note which model degrades most gracefully.
- **Accuracy versus variety.** Popularity recommends the same few titles to everyone, so its catalogue coverage is low; content-based lists can be narrow in genre. Read graph 10 alongside the ranking graphs before naming a winner.
- **Cost.** Collaborative filtering needs a user-by-user similarity matrix, so fit and scoring time grow with the number of users (graph 11).

**Recommendation to write up:** *[state which model you would deploy, for which kind of user, and why, using your average-rank result and the trade-offs above]*.

## Challenges and how they were solved

| Challenge | What went wrong | Solution |
|---|---|---|
| Unfair model comparison | The first evaluation scored all three models with one shared popularity ranking, so Precision, Recall and NDCG were identical for every model, and the "content" and "collaborative" rows did not use those methods | Rewrote the evaluation so each model makes its own rating predictions and its own top-K lists on the same split, with a random baseline for reference |
| Slow evaluation | Looping over users and movies row by row is too slow for 100,000 ratings | Vectorised the models with matrix operations (NumPy), so all users are scored in seconds |
| Many ties in content scores | Thousands of films share the same genre set, so a genre-only score cannot rank them | Added a tiny popularity term used only to break ties, and documented it |
| Sparse data for similarity | Most user-movie pairs are unrated, and a film seen by one neighbour can look perfect | Used the 40 nearest neighbours, mean-centred ratings, and a support-aware shrinkage in the ranking score |
| Application start-up error | The dashboard imported a function that did not exist | Removed the stale import and old evaluator, and tested every recommender path |

| Challenge | What went wrong | Solution |
|---|---|---|
| Showing results clearly | Dozens of metrics are hard to read in a table | Grouped graphs by question (error, classification, ranking, variety, speed, overall) with a consistent colour per model |

## Limitations and future work

### Limitations

- One random split on a small dataset; results change with the seed, so report them as indicative rather than exact.
- Ratings are explicit and the dataset is MovieLens, not Netflix; viewing behaviour and context (time, device) are absent.
- Ranking scores are low in absolute terms because each user has thousands of unseen films and only a few held-out likes; compare models with each other and with the random baseline.
- The evaluation uses vectorised versions of the recommenders, while the app's recommendation tables use the original implementations; the logic is aligned but not byte-identical.
- Genres are the only content signal; titles, tags and cast are unused.

### Future work

- Matrix factorisation (SVD) and item-based collaborative filtering, and a hybrid of content and collaborative scores.
- Repeated or time-based splits with confidence intervals.
- Use tags and release year as extra content features.
- Show a short "because you liked..." explanation next to each recommendation.
- Evaluate on the larger MovieLens datasets and add caching for speed.

## Conclusion

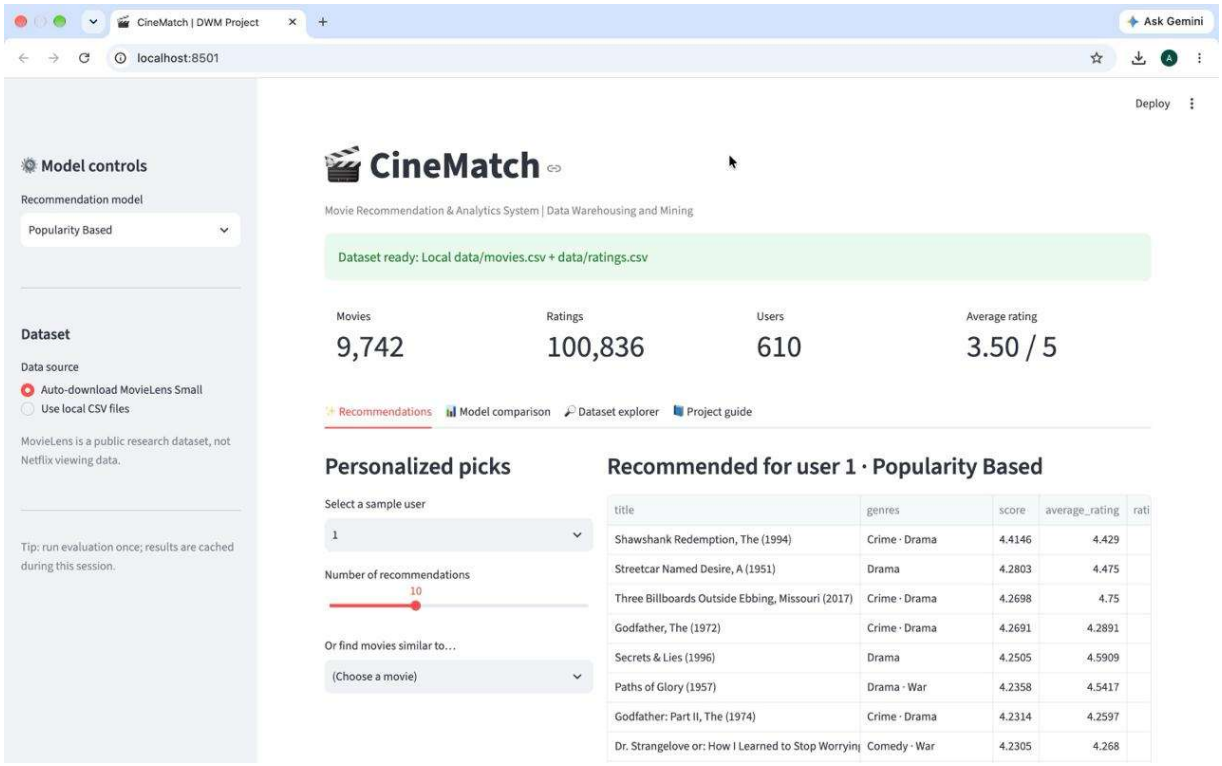
CineMatch shows the whole DWM workflow on a real problem: raw ratings are cleaned, turned into analytical tables, mined with three recommendation techniques, evaluated with a broad and fair set of metrics, and presented in an interactive dashboard. The comparison framework matters as much as the models: it makes the trade-offs between accuracy, ranking quality, variety and speed visible, so a model can be chosen for a reason.

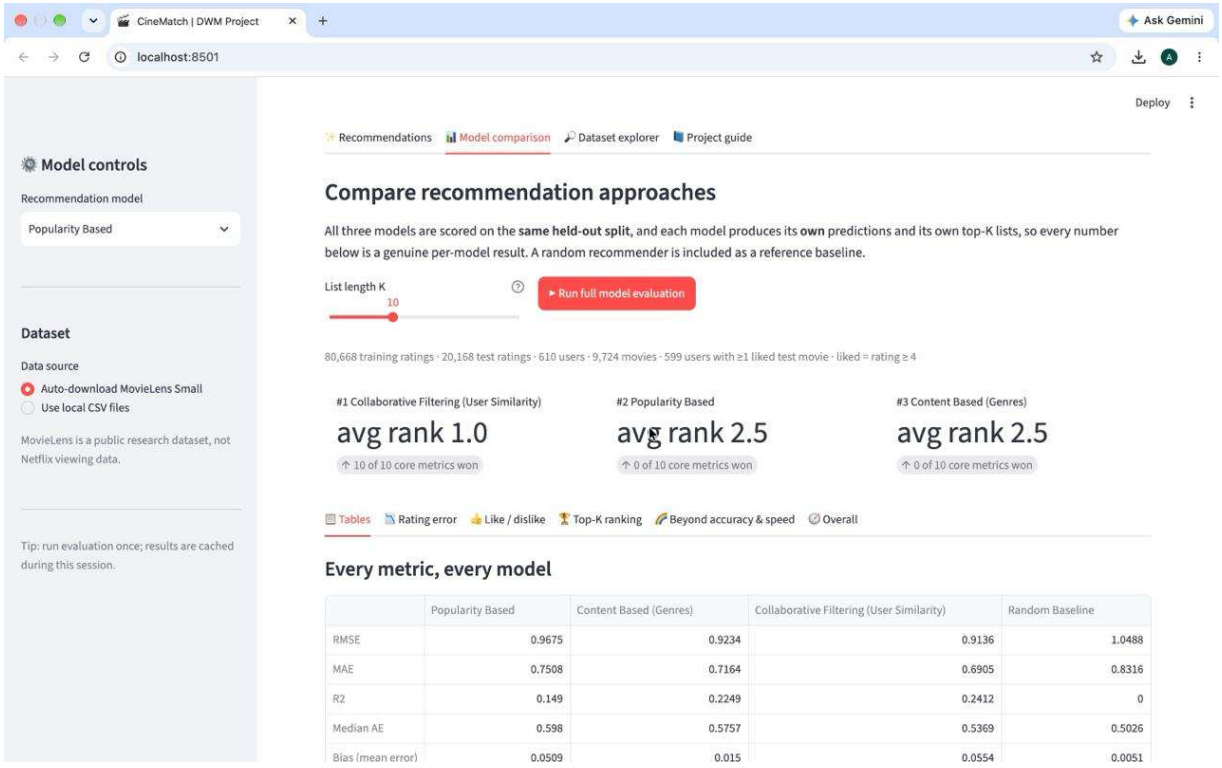
*[Add one or two sentences with your measured outcome, for example which model won and how it compared with the random baseline.]*

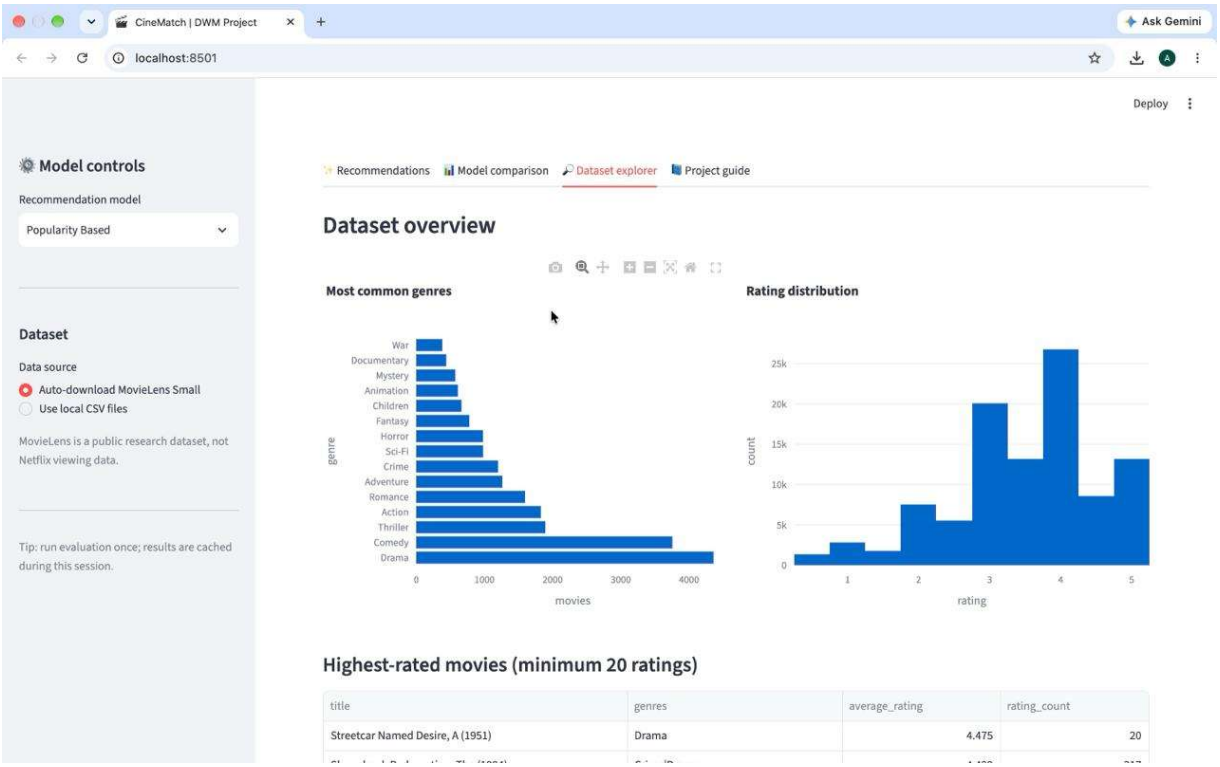
## References

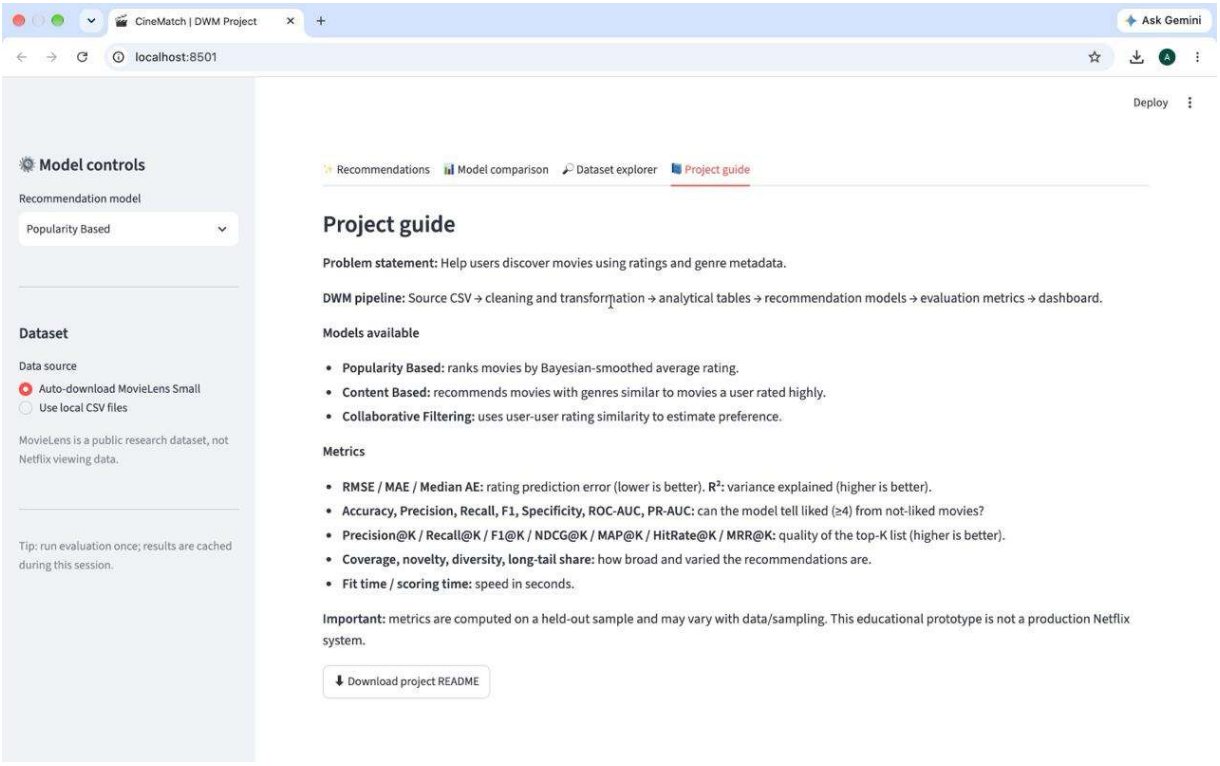

- Harper, F. M. and Konstan, J. A. (2015). The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems, 5(4), 19:1-19:19.
- MovieLens datasets, GroupLens Research

- Project source: `app.py`, `recommender.py`, `evaluation.py`, `plots.py`, `run_evaluation.py` (CineMatch_Movie_Recommendation_DWM)


CineMatch | DWM Project
Ask Gemini
localhost:8501
Deploy
Model controls
Recommendation model
Popularity Based
Dataset
Data source
Auto-download MovieLens Small
Use local CSV files
MovieLens is a public research dataset, not Netflix viewing data.
Tip: run evaluation once; results are cached during this session.
CineMatch
Movie Recommendation & Analytics System | Data Warehousing and Mining
Dataset ready: Local data/movies.csv + data/ratings.csv
Movies
9,742
Ratings
100,836
Users
610
Average rating
3.50 / 5
Recommendations
Model comparison
Dataset explorer
Project guide
Personalized picks
Select a sample user
1
Number of recommendations
10
Or find movies similar to…
(Choose a movie)
Recommended for user 1 · Popularity Based
title
genres
score
average_rating
Shawshank Redemption, The (1994)
Crime · Drama
4.4146
4.429
Streetcar Named Desire, A (1951)
Drama
4.2803
4.475
Three Billboards Outside Ebbing, Missouri (2017)
Crime · Drama
4.2698
4.75
Godfather, The (1972)
Crime · Drama
4.2691
4.2891
Secrets & Lies (1996)
Drama
4.2505
4.5909
Paths of Glory (1957)
Drama · War
4.2358
4.5417
Godfather: Part II, The (1974)
Crime · Drama
4.2314
4.2597
Dr. Strangelove or: How I Learned to Stop Worrying
Comedy · War
4.2305
4.268


CineMatch | DWM Project
Ask Gemini
localhost:8501
Deploy
Recommendations
Model comparison
Dataset explorer
Project guide
Model controls
Recommendation model
Popularity Based
Dataset
Data source
Auto-download MovieLens Small
Use local CSV files
MovieLens is a public research dataset, not Netflix viewing data.
Tip: run evaluation once; results are cached during this session.
Compare recommendation approaches
All three models are scored on the same held-out split, and each model produces its own top-K lists, so every number below is a genuine per-model result. A random recommender is included as a reference baseline.
List length K
10
▸ Run full model evaluation
80,668 training ratings · 20,168 test ratings · 610 users · 9,724 movies · 599 users with ≥1 liked test movie · liked = rating ≥ 4
#1 Collaborative Filtering (User Similarity)
avg rank 1.0
↑ 10 of 10 core metrics won
#2 Popularity Based
avg rank 2.5
↑ 0 of 10 core metrics won
#3 Content Based (Genres)
avg rank 2.5
↑ 0 of 10 core metrics won
Tables
Rating error
Like / dislike
Top-K ranking
Beyond accuracy & speed
Overall
Every metric, every model
Popularity Based
Content Based (Genres)
Collaborative Filtering (User Similarity)
Random Baseline
RMSE
0.9675
0.9234
0.9136
1.0488
MAE
0.7508
0.7164
0.6905
0.8316
R2
0.149
0.2249
0.2412
0
Median AE
0.598
0.5757
0.5369
0.5026
Bias (mean error)
0.0509
0.015
0.0554
0.0051


CineMatch | DWM Project
Ask Gemini
localhost:8501
Deploy
Model controls
Recommendation model
Popularity Based
Dataset
Data source
Auto-download MovieLens Small
Use local CSV files
MovieLens is a public research dataset, not Netflix viewing data.
Tip: run evaluation once; results are cached during this session.
Recommendations
Model comparison
Dataset explorer
Project guide
Dataset overview
Most common genres
War
Documentary
Mystery
Animation
Children
Fantasy
Horror
Sci-Fi
Crime
Adventure
Romance
Action
Thriller
Comedy
Drama
genre
0
1000
2000
3000
4000
movies
Rating distribution
25k
20k
15k
10k
5k
0
count
1
2
3
4
5
rating
Highest-rated movies (minimum 20 ratings)
title
genres
average_rating
rating_count
Streetcar Named Desire, A (1951)
Drama
4.475
20


CineMatch | DWM Project
Ask Gemini
localhost:8501
Deploy
Model controls
Recommendation model
Popularity Based
Dataset
Data source
Auto-download MovieLens Small
Use local CSV files
MovieLens is a public research dataset, not Netflix viewing data.
Tip: run evaluation once; results are cached during this session.
Recommendations
Model comparison
Dataset explorer
Project guide
Project guide
Problem statement: Help users discover movies using ratings and genre metadata.
DWM pipeline: Source CSV → cleaning and transformation → analytical tables → recommendation models → evaluation metrics → dashboard.
Models available
Popularity Based: ranks movies by Bayesian-smoothed average rating.
Content Based: recommends movies with genres similar to movies a user rated highly.
Collaborative Filtering: uses user-user rating similarity to estimate preference.
Metrics
RMSE / MAE / Median AE: rating prediction error (lower is better). R²: variance explained (higher is better).
Accuracy, Precision, Recall, F1, Specificity, ROC-AUC, PR-AUC: can the model tell liked (≥4) from not-liked movies?
Precision@K / Recall@K / F1@K / NDCG@K / MAP@K / HitRate@K / MRR@K: quality of the top-K list (higher is better).
Coverage, novelty, diversity, long-tail share: how broad and varied the recommendations are.
Fit time / scoring time: speed in seconds.
Important: metrics are computed on a held-out sample and may vary with data/sampling. This educational prototype is not a production Netflix system.
Download project README